COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/20/2026	**DATE SUBMITTED:**	8/20/2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 Linear and Binary Search

1. PROCEDURE OUTPUT

Introduction:

Searching is among the fundamental functions in data structures because applications frequently need to find certain information in a pool of data. For instance, an application could be required to find out the name of a student in a list, search for a certain number in an array, or look for a particular record in a database. Lack of a proper searching technique could make an application very inefficient, especially if there is lots of data.

As already mentioned, there are various searching algorithms that have pros and cons. Linear search and binary search are two of the simplest searching algorithms. In addition, there is the binary search tree (BST) data structure that is developed to enable efficient search of data.

Learning about this topic made me understand that searching is much more than looking for a certain value. It is the organization of data that makes such searches efficient.

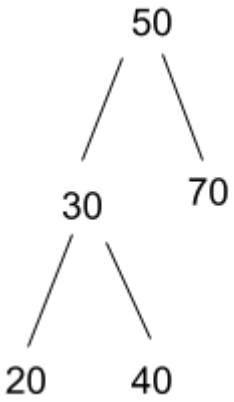
1. **What is a search tree in data structures?**

   The search tree is the tree structure that allows organizing values in certain order to allow efficient searching. The tree consists of nodes linked to one another. The top node is called the root. The nodes situated below some node are considered to be its children; the node situated above the others is called the parent node. A node without children is referred to as the leaf node.

   The Binary Search Tree (BST) is probably the most widely used example of the search tree. As the name suggests, in the binary search tree there are at most two children for each node. These are usually referred to as the left child and the right child. The rule of BST is simple:

   - Values smaller than the current node go to the left;
   - Values larger than the current node go to the right;
   - Values equal to the current node are processed differently depending on the implementation (may be ignored or put either to the left or to the right).

For example, if we insert the values 50, 30, 70, 20, and 40, the tree can look like this:



When we need to search 40, we start comparing it to 50. Since 40 is less than 50, we go to the left and arrive at 30. 40 being greater than 30 takes us to the right and 40 is found.

This is because of the fact that not only can we use comparisons but also eliminate certain data through comparisons. The whole concept of the search tree depends on this property.

Thus, the search tree is useful in situations where the data needs to be searched, deleted, or inserted repeatedly. But its efficiency largely depends upon the balance of the tree. A properly balanced binary search tree has a search complexity of O(log n) while an improperly unbalanced tree becomes like a linked list and has a complexity of O(n).

2. **What are the Different types of search algorithm in data structures? Differentiate each type of search.**
   There are different types of searching algorithms, but some of the common types include linear search and binary search. Other searching methods like hashing and tree searching methods are also used for particular cases.

   A. Linear Search
   Linear search is one of the most elementary searching methods, also referred to as sequential search method. Linear search algorithm involves checking the elements in the list or array sequentially until the required element is found or all the elements are exhausted.

   Suppose the following list is given:
   10 25 30 45 50

   The linear search will proceed by checking:
   10 → 25 → 30 → 45
   Since the required number was found, the algorithm will stop there.

   One benefit of linear search is that the list doesn't need to be sorted. The search can be performed on an unsorted list. However, the disadvantage is that the algorithm could take much time to search if there are too many elements.

   Worst-case time complexity for linear search is:
   O(n) **-** means the amount of work grows roughly **directly with the amount of data**.
   Thus, for 1,000 elements, it could take up to 1,000 steps to find the required number.

Binary search is a more efficient searching method, although it has one crucial condition – the data should be normally sorted. Rather than checking all the items, binary search checks the item at the middle and determines the half where the target could be located.

For instance:
10 20 30 40 50 60 70
If we search for 60, first we will check the middle item, 40.

As:
60 > 40
it is obvious that 60 is located to the right of 40. It means that we do not need to search the following items:
10 20 30 40

but only:
50 60 70
Then, we will check the middle item in the rest of the items and repeat this operation.

The main benefit is that during each iteration, binary search cuts the number of remaining items almost in half.
Worst-case scenario has the following time complexity:
O(log n) -means the algorithm becomes much more efficient because it **reduces the amount of data it needs to search after every step**.

Feature	**Linear Search**	**Binary Search**
Basic method	Checks elements one by one	Checks the middle and eliminates half
Data must be sorted?	No	Yes, normally
Speed	Slower for large data	Faster for large sorted data
Worst-case complexity	O(n)	O(log n)
Easy to implement?	Very easy	Slightly more complicated
Works with unsorted data?	Yes	No, unless the data is sorted first
Best use	Small or unsorted data	Large and sorted data

3. **What operations / implementations can be performed using binary and linear search operations?**
Linear Search and Binary Search algorithms are primarily used to locate a particular value from a set of values.

Linear Search
The following tasks can be performed using linear search algorithm:

Element – locating a particular element.
Position – identifying the location of the element.
Presence – locating whether the particular value is present in the set or not.
Unordered data – does not require the data to be sorted.
Duplicates – locating if the value is repeated within the set of values.

Example:
10, 20, 30, 40

In the above set if we have to locate 30, the linear search will go through the following process:
10 → 20 → 30

Binary Search
Binary search algorithm can be used for the following purposes:
Element
Position of an element
Presence
First/Last Occurrence
Correct Position of the value in the sorted data

But the binary search algorithm requires the data to be sorted.
Example:
10, 20, 30, 40, 50, 60, 70

Unlike linear search, instead of going through all the numbers, it will first check the middle one and then go left or right depending on the result.

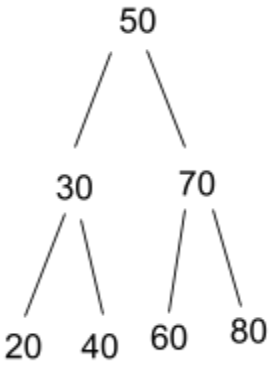
Linear Search looks at one item at a time, whereas Binary Search divides the search interval into two equal halves.

4. What are the advantages in using binary search trees as data structure?

Some advantages of a Binary Search Tree (BST) include:

1. Quicker Searches
The BST can provide quick searches since small numbers will be found on the left side, while large numbers will be found on the right.

Example:



For instance, if we want to search for 60, we begin at 50 and move to the right node to reach 70 and then to the left to reach 60.

2. Ease of Adding Data
Additional values may be added by adhering to the left and right rules of the tree.

3. Ease of Removing Data
In addition, removal of the data can be done while still maintaining the organization of the tree.

4. Organization of the Data
The BST automatically divides the smaller and larger data.

5. Creation of the Sorted Data
The use of in-order traversal enables the display of the data from the smallest to the largest.

For example:
20 → 30 → 40 → 50 → 60 → 70 → 80

6. Scalability
A BST has the ability to have its nodes increased or decreased as required without having a fixed size as in a simple array.

5. Give an example program using binary search and Linear search.

LINEAR SEARCH:

```cpp
#include <iostream>
#include <vector>
using namespace std;

int search(vector<int> &arr, int target){
    int n = arr.size();

    // Iterate linearly through the array
    for (int i = 0; i < n; i++)
        if (arr[i] == target)
            return i;
    return -1;
}

int main() {
    vector<int> arr = {2, 3, 4, 7, 1, 5};
    int target = 7;
    int index = search(arr, target);
    cout<<index<<endl;
    return 0;
}
```

OUTPUT:

```
3
PS C:\Users\Justine\Desktop\testiawe>
```

BINARY SEARCH:

```cpp
#include <iostream>
#include <vector>
using namespace std;

int binarySearch(vector<int> &arr, int target, int low, int high) {

    // Repeat until the pointers low and
    // high meet each other
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}

int main(){
    vector<int> arr = {2, 3, 4, 7, 9, 10};
    int n = arr.size();
    int target = 7;
    int low = 0;
    int high = n - 1;
    int index = binarySearch(arr, target, low, high);
    cout<<index<<endl;
    return  0;
}
```

OUTPUT:

```
3
PS C:\Users\Justine\Desktop\testiawe>
```

Conclusion

Understanding search trees, linear search, binary search and Binary Search Trees showed me that searching is more than just searching for a particular value. Organization of data impacts the efficiency of the programming task. This theme helped me compare the new concept with the previous data structures including stacks, queues and linked lists.

From the previous lessons, I know that stacks use LIFO (last in first out) principle while queues use FIFO (first in first out) principle. The linked lists differ from these two concepts because of the way their nodes are linked and allow insertion or deletion of nodes easily. Unlike those three data structures, Binary Search Tree emphasizes organizing data using values with smaller values to the left and larger values to the right.

Another concept was linear and binary searches. Linear search performs sequential search of an array while binary search makes it possible to divide search range in half and is efficient but requires sorted data. That gave me an understanding of the meaning of O(n) and O(log n).

Overall, this topic showed me that search trees and searching algorithms are designed for different ways of finding data. Linear search is simple but slow because it checks each item, while binary search is much faster but requires sorted data. The Binary Search Trees merge these concepts through arranging the values in a structured manner that makes search process faster, thus making the time complexity become O(log n). I still need some more practice on how to do binary search and Binary Search Tree, but doing the comparison between linear search, binary search, and BSTs made me realize the appropriate use of each.

REFERENCES:

Programiz. (n.d.). *Binary search*. https://www.programiz.com/dsa/binary-search

GeeksforGeeks. (2025, July 2). Linear Search vs Binary Search. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/linear-search-vs-binary-search/

W3Schools.com. (n.d.). https://www.w3schools.com/dsa/dsa_algo_linearsearch.php

Krishna, A. (2024, August 14). Search algorithms – linear search and binary search code implementation and complexity analysis. freeCodeCamp.org. https://www.freecodecamp.org/news/search-algorithms-linear-and-binary-search-explained/

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **QuillBot and Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for grammar correction and explaining concepts only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.